ENGENHARIA DE SOFTWARE I • PROJETO DE SISTEMAS

DOCUMENTO DE ARQUITETURA E ESPECIFICAÇÃO DE REQUISITOS

Sistema de Gestão Financeira Pessoal e Familiar • *Meu Bolso*

Alinhamento Acadêmico: *Capítulo 1 — Como produzir um software de qualidade elevada?*
Autor Base Teórica: *Prof. Igor Fernandes Menezes (EAD Laureate)*
Principais Referências: *Pressman (2016), Sommerville (2011), Larman (2007), Magela (2006), Teles (2004)*
Metodologia Adotada: *Métodos Ágeis (Scrum + XP) com Prototipagem Evolutiva*
Arquitetura de Software: *Camadas Desacopladas (SPA React + API RESTful Node/Express + PostgreSQL)*

Projeto: Meu Bolso (Finanças Fácil) • Versão 2.0
Data do Documento: Outubro de 2026
Status: Homologação Local / Documentação de Engenharia

1. Introdução e Contexto Acadêmico

A produção de um software de elevada qualidade exige a aplicação sistemática de práticas consolidadas da Engenharia de Software, superando abordagens amadoras conhecidas na literatura acadêmica como **'codifica-remenda'** (Menezes, p. 8; Boente, Moré e Cosenza, 2009). Conforme ensina Shari Pfleeger (2004, p. 2), 'como engenheiros de software, utilizamos nosso conhecimento sobre computadores e computação para ajudar a resolver problemas práticos'.

O presente documento formaliza a arquitetura, o modelo de ciclo de vida e a especificação de requisitos do sistema **Meu Bolso**, fundamentando cada decisão técnica nas teorias de Engenharia de Software ministradas no capítulo analisado, conectando-as à realidade prática de uma fábrica de software ágil moderna.

1.1 O Perigo do Modelo 'Codifica-Remenda' e a Pirâmide de Erro

O texto acadêmico alerta com rigor sobre o perigo de iniciar a codificação sem planejamento estruturado. De acordo com Magela (2006, p. 13) através da **Pirâmide de Propagação de Erro**, um requisito omitido ou mal compreendido acarreta um custo exponencialmente maior à medida que o projeto avança:

Fase do Ciclo de Vida	Fator de Custo Relativo de Correção (Magela, 2006)	Impacto no Projeto Meu Bolso
Requisitos / Concepção	1 – 2x	Definição clara das regras financeiras e entidades.
Análise	5x	Modelagem conceitual de fluxo de caixa e orçamentos.
Projeto / Arquitetura	10x	Definição do desacoplamento REST e contratos de API.
Implementação	20x	Codificação com TDD para evitar retrabalho de regras.
Testes	50x	Testes automatizados e homologação em ambiente local.
Manutenção (Produção)	**200x** (Crítico)	Evitado mediante entregas contínuas e validações prévias.

2. Comparativo dos Modelos de Ciclo de Vida (Menezes, 2018)

O capítulo acadêmico revisa os principais modelos de processo de software tradicionais e evolutivos. A tabela abaixo sintetiza as características analisadas pelo autor:

Modelo	Origem / Autor	Vantagens	Limitações / Riscos
Cascata Puro	Royce (1970)	Fases e marcos bem definidos; controle rígido.	Baixa visibilidade para o cliente; não permite retorno; alto risco de insatisfação.
Cascata c/ Retorno	Rodrigues (2005)	Permite recuar uma fase anterior para correções.	Risco de retrabalho elevado; difícil mensuração de esforço.
Espiral	Boehm / Sommerville (2011)	Excelente gestão de riscos; iterações contínuas.	Alta complexidade de gerenciamento; exige especialistas em risco.
Prototipagem Evolutiva	Rodrigues (2005)	Visibilidade imediata das telas; cliente valida requisitos reais.	Pode gerar ilusão de que o sistema está pronto apenas pelas telas.
Entrega Evolutiva	Rodrigues (2005)	Separa desenho arquitetônico das iterações funcionais.	Exige arquiteto sênior no início e boa divisão de escopo.

RUP (Rational)	IBM / Rational (1998)	Processo completo com 4 fases e 9 disciplinas.	Pesado para equipes enxutas; alto custo de artefatos.

3. Metodologia Ágil Selecionada: Justificativa

Conforme indicado na síntese do material acadêmico (p. 22-23), para projetos contemporâneos em ambientes dinâmicos com requisitos evolutivos, a abordagem mais eficaz é a combinação de **Métodos Ágeis** com **Prototipagem Evolutiva**. O sistema *Meu Bolso* adota um modelo híbrido estruturado:

1. Gestão por Scrum (Sommerville, 2011):
• *Sprints curtas (2 semanas):* Entregas de incrementos funcionais (software operável a cada ciclo).
• *Product Backlog priorizado por MoSCoW (Oliveira, 2014):* 'Tem que fazer' (Must), 'Deve fazer' (Should), 'Poderia fazer' (Could) e 'Interessante fazer' (Won't).
• *Reuniões diárias (Daily Scrum):* 10 a 15 minutos em pé para identificação rápida de impedimentos.

2. Engenharia Técnica por Extreme Programming - XP (Teles, 2004):
• *Desenvolvimento Orientado a Testes (TDD):* Criação do teste antes do código, garantindo qualidade intrínseca.
• *Refatoração Contínua:* Limpeza sistemática do código sem alteração do comportamento externo.
• *Pequenas Versões e Integração Contínua:* Deploy em ambiente de homologação local antes de produção.
• *Posse Coletiva e Padrões de Código:* Convenções limpas mantidas por toda a equipe.
• *Ritmo Sustentável:* Produtividade contínua evitando débitos técnicos.

4. Arquitetura de Software Ideal para Métodos Ágeis

Para suportar iterações ágeis frequentes sem gerar quebras estruturais, a Engenharia de Software preconiza uma **Arquitetura em Camadas Desacoplada (Decoupled Layered Architecture)**, operando sob o paradigma **Client-Server RESTful**. Essa arquitetura maximiza a independência dos times e atende aos princípios de flexibilidade e manutenção.

4.1 Visão Estrutural das Camadas do Sistema

Camada	Tecnologias	Papel no Contexto Ágil & Engenharia
1. Apresentação (Frontend SPA)	React 18 • Vite 7 • Tailwind CSS • Lucide • Recharts	Materializa a *Prototipagem Evolutiva* (Rodrigues, 2005). As telas são protótipos funcionais que evoluem diretamente para código final. Comunica-se de forma assíncrona via JSON com a API.
2. Fachada de API (Controllers / Gateway)	Node.js • Express REST API • Middlewares (JWT, CORS)	Expõe contratos estáveis (Endpoints REST). Permite mock de dados durante o desenvolvimento ágil, acelerando testes de tela enquanto o backend implementa as regras.
3. Domínio / Lógica de Negócios (Services Layer)	Módulos JS isolados • Regras de Cálculo Financeiro • DRE • Câmbio	Concentra as *Regras de Negócio (RN)* puras, independentes de interface gráfica ou banco. Viabiliza TDD (XP) com testes unitários de alta velocidade.
4. Persistência de Dados (Data Access Layer)	PostgreSQL • Supabase • Pool de Conexões • Migrations	Garante integridade referencial ACID para contas, cartões e transações. Possibilita rotinas de backup, auditoria e rollback automático.

4.2 Por Que Esta Arquitetura é Superior em Métodos Ágeis?

1. Desacoplamento Front-End / Back-End: O front-end e o back-end possuem ciclos de deploy e desenvolvimento autônomos. A equipe de UX/UI pode refinar temas, mascote FIN e dashboards sem tocar nas regras de transações do banco.
2. Testabilidade Extrema (TDD de XP): Ao isolar a camada de serviços (Service Layer), testes automatizados validam cálculos de DRE, juros e orçamentos em milissegundos sem depender de navegador aberto.
3. Suporte Multiplataforma: A mesma API RESTful que alimenta a Web alimenta o aplicativo mobile (Android/Capacitor), eliminando duplicidade de regras de negócio.

5. Engenharia de Requisitos Aplicada (Menezes, p. 20-21)

Conforme estabelecido nas páginas 20 e 21 do material de Engenharia de Software I, um requisito de software possui uma sintaxe formal estrita para garantir conformidade e eliminar ambiguidades:

- **Identificação do Sistema:** O Sistema no qual deve ser construída a funcionalidade.
- **Ator:** Quem provoca estímulo (Pessoa, Sistema Externo, Periférico).
- **Funcionalidade / Ação:** O que deve ser executado pelo sistema.

5.1 Requisitos Funcionais (RF) com Priorização MoSCoW

ID	Descrição Formal do Requisito Funcional	MoSCoW
RF01	O **Sistema Meu Bolso** deve permitir que o **Usuário** realize cadastro, autenticação (e-mail/senha ou Google) e recuperação de acesso.	**Must**
RF02	O **Sistema Meu Bolso** deve permitir que o **Usuário** registre, edite e categorize receitas financeiras.	**Must**
RF03	O **Sistema Meu Bolso** deve permitir que o **Usuário** registre, edite e categorize despesas financeiras com forma de pagamento.	**Must**
RF04	O **Sistema Meu Bolso** deve permitir que o **Usuário** gerencie cartões de crédito, limites, faturas e parcelamentos.	**Must**
RF05	O **Sistema Meu Bolso** deve permitir que o **Usuário** estabeleça orçamentos mensais e acompanhe metas de economia.	**Should**
RF06	O **Sistema Meu Bolso** deve permitir que o **Usuário** visualize relatórios gráficos analíticos de fluxo de caixa.	**Should**
RF07	O **Sistema Meu Bolso** deve permitir que o **Usuário** emita o demonstrativo contábil DRE (Demonstrativo do Resultado).	**Should**
RF08	O **Sistema Meu Bolso** deve permitir que o **Usuário** planeje e controle orçamentos de viagens em múltiplas moedas.	**Could**
RF09	O **Sistema Meu Bolso** deve permitir que o **Usuário** consulte cotações de moedas internacionais em tempo real.	**Could**
RF10	O **Sistema Meu Bolso** deve permitir que o **Usuário** interaja com o mascote educacional FIN para auxílio e dicas.	**Could**
RF11	O **Sistema Meu Bolso** deve permitir que o **Usuário** realize backup criptografado e restauração dos seus dados.	**Should**
RF12	O **Sistema Meu Bolso** deve permitir que o **Usuário** alterne entre os temas *Preto com Dourado* e *Branco com Dourado*.	**Should**

5.2 Requisitos Não Funcionais (RNF) — Qualidade de Software

Conforme Sommerville (2011, p. 73), requisitos não funcionais são restrições e propriedades de qualidade do sistema:

ID	Categoria	Especificação do Requisito Não Funcional
RNF01	Segurança	O **Sistema Meu Bolso** deve proteger senhas com hash BCrypt (salt > 10) e utilizar tokens JWT com expiração de sessão.
RNF02	Desempenho	O **Sistema Meu Bolso** deve carregar o Dashboard e responder às requisições da API em tempo inferior a 1,5s sob conexões normais.
RNF03	Portabilidade	O **Sistema Meu Bolso** deve operar em navegadores modernos (Chrome, Edge, Firefox, Safari) e em dispositivos móveis via Capacitor/PWA.
RNF04	Usabilidade	O **Sistema Meu Bolso** deve manter suporte a dois temas de alto contraste (Preto c/ Dourado e Branco c/ Dourado) com feedbacks de ação via Toasts.
RNF05	Confiabilidade	O **Sistema Meu Bolso** deve realizar operações financeiras sob transações atômicas no banco com rollback em falhas.

5.3 Regras de Negócio (RN) — Domínio Financeiro

Para Larman (2007, p. 85), regras de negócio transcendem a tecnologia e pertencem às políticas do domínio da aplicação:

ID	Nome da Regra	Descrição da Regra de Negócio
RN01	Valor Positivo	Nenhum lançamento de receita ou despesa pode possuir valor monetário zerado ou negativo no ato da inserção.
RN02	Vínculo Obrigatório	Toda transação deve obrigatoriamente estar vinculada a um usuário cadastrado e a uma categoria financeira válida.
RN03	Consistência de DRE	O cálculo do Demonstrativo DRE deve deduzir despesas operacionais da receita operacional líquida segundo normas de contabilidade gerencial.
RN04	Alerta de Limite	Ao cadastrar despesa em cartão cujo total ultrapasse o limite definido, o sistema deve alertar o usuário sobre o estouro de fatura.
RN05	Segurança de Senha	Senhas devem possuir no mínimo 6 caracteres para atendimento à política de proteção básica de contas.

6. Estrutura Física dos Módulos do Sistema

A estrutura de pastas do projeto reflete fielmente o desacoplamento de responsabilidades e as boas práticas de Clean Code:

Caminho / Diretório	Tipo / Camada	Responsabilidade no Sistema
src/components/	UI / Apresentação	Componentes reutilizáveis (LayoutNovo, Mascote FIN, Toaster, ThemeProvider).
src/pages/	Visões / Telas	Telas do sistema: Dashboard, Receitas, Despesas, Cartões, Orçamento, DRE, etc.
src/contexts/	Estado Global	Contextos de autenticação (AuthContext), moedas (CurrencyContext) e layout.
src/services/api.js	Integração REST	Cliente Axios/Fetch para comunicação com os endpoints da API backend.

server/ ou api/	Backend / Serviços	Controladores, rotas de autenticação, serviços de banco de dados PostgreSQL.
android/	Mobile Container	Configuração Capacitor para build nativo Android a partir da mesma SPA.
docs/	Engenharia	Documentação arquitetural, guias de migração e especificações de software.

7. Conclusão e Recomendações

A fundamentação do projeto **Meu Bolso** nos princípios de Engenharia de Software I comprova que a união de **processos ágeis (Scrum/XP)** com uma **arquitetura desacoplada em camadas** neutraliza os riscos da abordagem 'codifica-remenda', garante aderência às necessidades reais dos usuários através de protótipos evolutivos e assegura baixo custo de manutenção ao longo de todo o ciclo de vida do software.

8. Referências Bibliográficas (Material Acadêmico)

• **BOENTE, A.; MORÉ, J.; COSENZA, H.** Avaliação Fuzzy da qualidade de produtos de software numa fundação pública estadual. VII SIMPEP, 2009.
• **LARMAN, C.** Utilizando UML e padrões: uma introdução à análise e ao projeto orientado a objetos e ao desenvolvimento iterativo. 3. ed. Porto Alegre: Bookman, 2007.
• **MAGELA, R.** Engenharia de Software aplicada – fundamentos. Rio de Janeiro: Alta Books, 2006.
• **MENEZES, I. F.** Engenharia de Software I: Como produzir um software de qualidade elevada? EAD Laureate Brasil, 2018.
• **OLIVEIRA, R. R.** A Técnica de Priorização MoSCoW. Management Plaza International, 2014.
• **PFLEEGER, S. L.** Engenharia de Software – Teoria e Prática. 2. ed. São Paulo: Pearson, 2004.
• **PRESSMAN, R.** Engenharia de Software: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.
• **RODRIGUES, C. Á.** Engenharia de Software. Apostila de Especialização, Centro Universitário UNA, 2005.
• **SOMMERVILLE, I.** Engenharia de Software. 9. ed. São Paulo: Pearson Addison Wesley, 2011.
• **TELES, V. M.** Extreme Programming: aprenda como encantar seus usuários desenvolvendo software com agilidade e alta qualidade. São Paulo: Novatec, 2004.